RAILSAFE

Backend API & Database Schema

Descriptive reference to the current FastAPI routes, Pydantic request schemas, serializers and SQLAlchemy models.

API base: /api **Development docs:** http://localhost:8000/docs

1. API overview

All routes listed here are prefixed with /api. Protected routes require a bearer JWT. Sensor and ML ingest routes use their configured API-key authentication. List endpoints generally return JSON objects containing items, with pagination/filter query parameters where supported. Error responses use the application error envelope; request validation errors are 4xx.

Authentication and users

Method / route	Purpose and schema
POST /auth/login	LoginRequest: username, password. Returns token, role, name, worker_id, city, source.
POST /auth/register	RegisterRequest: name, email, password (10+), optional city and requested role. Public registration creates a worker request; elevated role is controlled by server policy.
GET /auth/me	Returns the authenticated PublicUser.
POST /auth/forgot-password	ForgotPasswordRequest: email. Returns generic accepted response.
POST /auth/reset-password	ResetPasswordRequest: token, password (10+).
GET /users	Admin account list; query filters/pagination.
GET /users/pending	List pending worker registrations.
POST /users/{user_id}/approve	Approve worker signup; no body.
POST /users/{user_id}/reject	Reject worker signup; no body.
POST /users/{user_id}/reset-password	AdminPasswordResetRequest: password (10+).
GET /workers	List approved workers with role/city scoping.

Alerts and tasks

Method / route	Purpose and schema
GET /alerts	Paginated/filterable alerts (city, status, severity, time, search).
GET /alerts/{id}	Alert detail via alert serializer.
GET /alerts/{id}/image	Stored alert image bytes.
GET /alerts/{id}/history	Audit history for one alert.
POST /alerts/{id}/acknowledge	ActionNotes: optional notes.
POST /alerts/{id}/resolve	ActionNotes; also closes/cancels its open task transactionally.
POST /alerts/{id}/false-positive	ActionNotes; also closes/cancels open task.
POST /alerts/{id}/assign	AssignAlertRequest: worker_id, optional notes.
POST /alerts/{id}/priority	AlertPriorityRequest: severity S1/S2/S3.
GET /tasks	List tasks, filters include status/worker and pagination.
POST /tasks	CreateTaskRequest: alert_id, worker_id, status=ASSIGNED, optional notes.
POST /tasks/{id}/assignment	Assign/reassign an unstarted task; worker_id.
DELETE /tasks/{id}/assignment	Unassign worker.
PUT /tasks/{id}/status	TaskStatusRequest: ACCEPTED, IN_PROGRESS or COMPLETED; optional notes.
POST /tasks/{id}/submit	Submit task work for review; ActionNotes.
POST /tasks/{id}/review	TaskReviewRequest: approve or request_changes, optional notes.
POST /tasks/{id}/evidence	Multipart evidence file plus notes/evidence_type.
GET /tasks/{id}/evidence/{evidence_id}	Fetch authorized evidence.

2. Device, telemetry and ML contracts

Method / route	Purpose and schema
POST /integrations/sensors/readings	SensorReadingsBatch: device_id, source (esp32/hardware/raspberry_pi), readings (1-100). Each reading: name, JSON value, status, optional unit/interpreted_value/recorded_at/lat/lon.
POST /devices	DeviceRegistration: id, name, optional platform and city.
GET /devices	List devices and sync/diagnostic status.
GET /devices/{id}/sensors	Return latest sensor readings per device.
GET /devices/{id}/diagnostics	Diagnostic JSON and recent inference details.
GET /devices/{id}/camera/latest	JPEG bytes for latest frame if present.
GET /tracks	Tracks with geometry/status/occupancy; city/filter/pagination.
GET /trains	Train position/status; city/filter/pagination.
GET /switch/status	Current switch-decision status.
POST /switch/decision	SwitchDecisionRequest: action, confidence, optional timestamp/train_id/switch_id/target_track/reason/coords, rules_checked and raw_scores.
GET /dashboard/summary	Aggregated counters from database GROUP BY.
GET /dashboard/event-log	Event log; supports since and pagination.
GET /model/status	Admin model readiness/status.
GET /ai/detection	Admin latest detection data.
POST /integrations/ml/events	JSON MlEventRequest; idempotent event ingest.
POST /integrations/ml/camera/frame	Multipart device_id + JPEG + optional captured_at/lat/lon; runs local YOLO.
POST /integrations/ml/events/with-image	Multipart event JSON and image, size-limited and authenticated before body processing.
POST /integrations/ml/frames	Validated, size-capped frame/diagnostic payload.

MlEventRequest fields

schema_version="1.0"; source_event_id; detected_at (timezone required); source; device_id; defect; severity S1/S2/S3; confidence 0..1; optional latitude, longitude, gps_accuracy_m, chainage, track, boxes (max 100), sensor_fusion object, model_version, inference_ms and city. Box coordinates are [x1,y1,x2,y2]. Duplicate source-event IDs are deduplicated.

Example sensor batch

```
{"device_id":"esp32-cam-01","source":"esp32","readings":[{"name":"ultrasonic","value":42.5,"status":
"OK","unit":"cm","recorded_at":"2026-11-08T10:00:00Z","latitude":12.97,"longitude":77.59}]}
```

3. Database schema and relationships

Table	Important columns / purpose
users	id PK; name; email unique; username unique; password_hash; role; approval_status; city; worker_id unique; is_active; auth_version; source; timestamps.
password_reset_tokens	id PK; user_id FK; token_hash unique; expires_at; used_at; created_at.
alerts	id PK; defect; severity; status; confidence; track/chainage; coordinates; gps_metadata JSON; sensors JSON; image_url; source + source_event_id unique; city; assigned worker FK; detected_at; timestamps.
alert_history	id PK; alert FK; actor FK; action; old/new status; changes JSON; notes; source; timestamp.
devices	id PK; name; platform; city; status/network status; diagnostics/pi_status JSON; last_sync; source; timestamps.
sensor_readings	id PK; device FK; name/status; numeric/text/structured value; unit/interpreted value; x/y/z; vibration/anomaly; GPS metadata; source; recorded_at.
maintenance_tasks	id PK; alert FK; worker FK; status; priority; admin/review notes; review state/time; source; timestamps.
task_evidence	id PK; task FK; stored filename unique; original filename; MIME; byte_size; evidence_type; notes; uploader FK; created_at.
tracks	id PK; name/status/occupancy; max_speed; severity; geometry JSON; city/source; timestamps.
trains	id PK; latitude/longitude; speed; destination; next_switch; distance; GPS JSON; city/source; measured_at; timestamps.
decision_states	id PK; action/status; train FK; switch_id; target track; confidence; reason; coordinates; rule checks/scores/payload JSON; source; updated_at.
event_log	id PK; actor FK; action; object type/id; summary; details JSON; source; timestamp.
ml_origin_events	id PK; unique source_namespace + source_event_id; device/alert FKs; model_version; inference_ms; payload JSON; detected/created times.

Relationships and constraints

Users can own assigned alerts/tasks, upload evidence and author audit records. Devices own sensor readings and ML events. Alerts own history, tasks and ML event links. Tasks belong to one alert and can contain multiple evidence items. Trains may be referenced by decision states. Foreign keys use CASCADE for dependent readings/evidence, SET NULL for optional user/device links, and RESTRICT from task to alert. A partial unique index enforces one unfinished task per alert. Severity values are S1/S2/S3; task states are ASSIGNED, ACCEPTED, IN_PROGRESS, AWAITING_REVIEW, COMPLETED.

Current implementation notes

The camera endpoint currently loads one configured detector and runs inference on CPU. It currently assigns S2 to camera detections; geometry-based S1/S2/S3 severity is not implemented. If the model is trained as a single crack class across wood and metal, the detector can return crack boxes for either surface, but material-independent severity rules still need to be defined and implemented.

Source code map

SQLAlchemy: backend/app/models/entities.py. Request models: backend/app/schemas/auth.py, operations.py, telemetry.py, ml.py. Response serializers: backend/app/schemas/serializers.py. Routers: backend/app/api/routers/auth.py, operations.py, telemetry.py, users.py, ml_ingest.py. Fresh database: run the repository Alembic migrations before starting the API. The live OpenAPI document is the final authority for exact fields and response models.